

1

a

This reduction is quite similar to the weighted interval scheduling to shortest paths.

We start with creating n nodes and we are trying to find the shortest path from node 1 to node n . We can attach arcs from node i to node $i + 1$ with cost 0 to ensure a feasible solution. Then for each arc (i, j) in the arc set A , we put an arc from node i to node $j + 1$ (with the exception of $j = n$, in which case the arc goes to the node n) with a cost of -1 . Thus when trying to find the shortest path, the shortest path algorithm will take as many of these arcs with cost -1 which is the same as trying to find the most amount of arcs in a set that do not conflict.

We know arcs are non conflicting in the shortest paths algorithm because in order for an arc (i_1, j_1) to be conflicting with another arc (i_2, j_2) then $i_1 \leq i_2 \leq j_1 \leq j_2$. In the case of our graph construction, it is not possible to take arcs that satisfy these inequalities because the only way for a path to take 2 arcs that conflict is if $j_1 = i_2$, which is why the arcs go from i_1 to $j_1 + 1$ to prevent a path from taking conflicting arc. In other words, we need the $j + 1$ because unlike the WIS problem we are not allowed to have an arc that stops and then use another arc on the same node. Thus the $+1$ for the ending part of an arc forces the paths to only take arcs once it is no longer conflicting with the previous arc taken.

Additionally given the extra constraint of $\forall$ arcs, $i \leq j$, then we know that taking any path allows increases our node number essentially moving “forward” so we know that we never have a cycle allowing for a negative infinite path. To finish the reduction, the optimal objective would be equal to the most amount of non conflicting arcs in a set (times -1). The actual arcs chosen would be the arcs $i, j + 1$ that are part of the shortest path.

b

The problem from HW1 states that the input is “given a set A of n -arcs”, thus as long as the algorithm runs in polynomial time with respect to $|A|$ then the algorithm would run in polynomial time. Thus in our case n is a constant and not actually the size of the input.

Our algorithm would run similarly to the shortest paths reduction above, except we need to account for cases in which the n -arcs loop around (ie $j < i$). We can do this by running a series of shortest paths algorithms (which was discussed to be polynomial time) on n different graphs. Consider a graph of n vertices that are connected sequentially (node i has an outgoing arc to node $i + 1$) with an additional arc from node n to node 1. All of these arcs have 0 cost. Next, add the n -arcs onto our graph with the same rules as in 1.a which is an n -arc (i, j) creates an arc from node i to node $j + 1$ with a cost of -1 .

Now for determining the maximal non-conflicting arc set problem, we run the shortest paths algorithm on slightly modified versions of this graph. We start by adding a new target node t

with each run of the shortest paths, from an original node i to t and all edges that are incoming to i we instead point to t (including the 0 cost edges). To prevent use of any conflicting arcs from both being used in the path, we remove any edge that effectively points “backwards” in our line (since we essentially unwrapped the circle into different sequential lines). This runs the shortest paths algorithm n times and then the maximal non-conflicting arc set corresponds to the shortest path of all of the paths on the n different graphs.

Note that an edge that points “backwards” in one of these modified graph does not point “backwards” in all modified graphs. Suppose there is an arc where $j + 1 \leq i$, then if our start node s satisfies the inequalities $j + 1 \leq s \leq i$ then this arc would appear to point forward in this line and could be used in an optimal shortest path for this graph. By doing these different graph splits, essentially we are shifting over all the indices of the arcs in the circle looking for an optimal one.

Thus the runtime is $O(nS + G)$ where n is from the n -arcs, S is the runtime for the shortest paths algorithm, and G is the time for constructing the graph. We know that $G = O(n + m)$ where $m = |A|$ because that is the amount of time it takes to create the edges.

2

a

We can better visualize this problem as Ay where the individual rows of this matrix add up to u and the individual columns add up to v . Specifically the sum of the first row of Ax should be equal to u_1 and so on (same with columns). Now our goal is to prove that if Bx has a solution (given the same row and column summation constraints) then Ay must also have a solution.

This is quite simple since $\forall a_{ij} \neq 0, b_{ij} = 1 = a_{ij}/a_{ij}$. Thus, $b_{ij}x_{ij} = a_{ij}y_{ij} \rightarrow x_{ij} = a_{ij}y_{ij}$. This makes intuitive sense because $b_{ij} = 0$ when $a_{ij} = 0$, meaning that the values of x_{ij}, y_{ij} cannot contribute to their respective sums. On the other hand, when $b_{ij}, a_{ij} \neq 0$ the steps shown before how to solve for y_{ij} or x_{ij} from the other.

More formally, if $\exists x$ that is feasible, then $\forall i \in p, \sum_{j=1}^q b_{ij}x_{ij} = u_i \wedge \forall j \in q, \sum_{i=1}^p b_{ij}x_{ij} = v_j$. Since B and A have the same 0 entries, we ignore those respective x and y . For any nonzero entry, we apply the same formula above to find $x_{ij} = a_{ij}y_{ij}$ will result in a feasible solution because $\forall i \in p, \sum_{j=1}^q b_{ij}x_{ij} = u_i \wedge \forall j \in q, \sum_{i=1}^p b_{ij}x_{ij} = v_j \rightarrow \forall i \in p, \sum_{j \in q \wedge b_{ij} \neq 0} x_{ij} = u_i \wedge \forall j \in q, \sum_{i \in p \wedge b_{ij} \neq 0} x_{ij} = v_j \rightarrow \forall i \in p, \sum_{j=1}^q a_{ij}y_{ij} = u_i \wedge \forall j \in q, \sum_{i=1}^p a_{ij}y_{ij} = v_j$. Thus, if $\exists x$ that is feasible then $\exists y$ that is feasible. Additionally the proof is very similar, but mirrored for if $\exists y$ that is feasible then there exists a feasible x .

Note the last $\rightarrow$ is because B and A have the same 0 entries.

For the other direction, if $\exists y$ that is feasible then $\exists x$ that is feasible. This proof is quite simple with a proof by contradiction, suppose there is no feasible y but there is a feasible x , then the steps above would outline how to find a feasible y from our feasible x . This creates a contradiction because we started with there not existing a feasible y . Therefore this creates a contradiction. Similar steps to show that if $\exists x$ feasible then $\exists y$ feasible.

Thus x has a feasible solution in those system of equations iff y has a solution in the original system of equations.

b

We can construct the graph as a bipartite one where V_1 consist of p nodes and V_2 consists of q nodes. The nodes in V_1 correspond to the rows of Bx and the nodes in V_2 correspond to the columns of Bx . We assign arcs from each node in V_1 to each node in V_2 . These arcs then represent x_{ij} where i is the row and j is the column. By our feasibility shown in **2.a**, we know that these flows x_{ij} can correspond to a feasible solution y_{ij} . These arcs have a lower or upper bound, since this is a circulation feasibility problem we can assume all 0 costs.

To induce a flow, we can create a new source node which has outgoing arcs to all the nodes in V_1 and all the nodes in V_2 point to the source node. Each of the outgoing arcs from the source to a node $i \in V_1$ has a lower bound of u_i , and similarly an arc from node $j \in V_2$ to the source node has a lower bound of v_j . Since we are given that $\sum_{i=1}^p u_i = \sum_{j=1}^q v_j$ we know that these ingoing and outgoing arcs will lead to a 0 circulation.

Since the circulation problem ends with all nodes having a 0 supply excess, we know that since each node $i \in V_1$ have an excess of u_i so $u_i - \sum_{j=1}^q x_{ij} = 0$. Similarly, since each node $j \in V_2$ have a deficit of v_j so $-v_j + \sum_{i=1}^p x_{ij} = 0$. Thus if there is a feasible solution then the values of x_{ij} directly correspond to a feasible solution of x_{ij} in **2.a** which was discussed to prove a feasible solution in the original problem.

3

The scheduling reduction fails quite similarly to the one discussed in class, except we also have to partition the intervals on the differing machines active. Thus we can compute the total compute power of an interval as the length of the interval multiplied by the number of machines in that interval (constant since we partitioned on machine intervals).

i	1	2	3	4
p_i	1.5	1.25	2.1	3.6
r_i	3	1	3	5
r_i	5	4	7	9

With machines as

Interval	Num Machines
[1,3]	3
[3,4]	2
[4,6]	3
[6,7]	2
[7,11]	4

We build off of these intervals to now partition on the different intervals of jobs active during those times and the total processing time at each interval

Interval	Num Machines	Jobs Active	Interval Length	Max Processing Power
[1,3]	3	2	3	9
[3,4]	2	1,2,3	1	2
[4,5]	3	1,3	1	3
[5,6]	3	3,4	1	3
[6,7]	2	3,4	1	2
[7,9]	4	4	2	8

Now on each of these intervals we look at how many jobs are in excess of the number of machines. For any other interval we are able to string jobs like in the original WIS problem where you start processing a job during this interval, once this one ends (or we reach the end of the interval), we start the processing the next job. If this subsequent job reaches the end of the interval then we restart at the beginning of the interval with the previous steps as before. Since for these intervals we have the constraint that the number of machines is greater than or equal to the number of jobs on this interval, we know that in this interval we either process a job for the length of the interval OR complete the job. We also know there is no overlap of a machine running the same job as another machine at the same time (ie running in parallel) because for each job we process at most the length of the interval with the same stacking method as shown in class.

Thus we can create a new table indexed on the intervals and the columns represent the processing time occurred for the corresponding job in that interval (after only the steps above).

Interval	p_1 in int	p_2 in int	p_3 in int	p_4 in int
[1,3]	0	1.25	0	0
[3,4] (skip for now)	0	0	0	0
[4,5]	1	0	1	0

Interval	p_1 in int	p_2 in int	p_3 in int	p_4 in int
[5,6]	0	0	1	1
[6,7]	0	0	0.1	1
[7,9]	0	0	0	1.6

Essentially now the algorithm would find out which jobs are remaining and which intervals still have available compute power. Thus we would construct a new WIS on varying number of uniform parallel machines. However our recursive problem would be slightly different where some jobs have non continuous intervals where they can be processed (ie not one start and stop time). This is because a job could be part of two disjoint intervals where the number of jobs was greater than the number of machines. Thus in our case our new jobs are:

i	1	2	3	4
p_i	0.5	0	0	0
intervals	[3,4]	[2.25,4]	[3,4],[6.1,9]	[8.4,9]

With machines, we need to subtract the number of jobs that used the full interval from the number of machines in that interval. If a job did not use the full interval and instead ended then we would partition the interval on the amount of processing that happened on that job (note that this means after the partition part of the interval could have 0 machines available while the other part would be 1).

Interval	Num Machines
[1,2.25]	2
[2.25,3]	3
[3,4]	2
[4,6]	1
[6,6.1]	0
[6.1,7]	1
[7,8.6]	3
[8.6,11]	4

Now the final step of this iteration is again to partition the intervals on machine intervals and job intervals. Here tho when listing jobs in an interval we skip any jobs with 0 processing time remaining.

Interval	Num Machines	Jobs Active	Interval Length
[3,4]	2	1	1

Thus our algorithm ends here with a machine processing the 0.5 remaining time for job 1. Although this is not the case for this example, the algorithm would repeat until there are intervals with jobs that still need to be processed but no more compute available.